End-to-End CI/CD Pipeline for a Calculator REST API

Subhash H

GitHub: github.com/Subhashhari/calculator-devops

Docker Hub: hub.docker.com/r/subhashh2005/calculator-api

1. Overview and pipeline architecture

A Spring Boot calculator API is taken from a pull request to a verified, load-balanced deployment with no manual step. Merging into `main` makes GitHub send a webhook to Jenkins. Jenkins runs the declarative pipeline in the repository's `Jenkinsfile`: it builds and tests the code, pushes a new image to Docker Hub, redeploys the Docker Compose stack, checks that the stack is healthy, and emails the result.

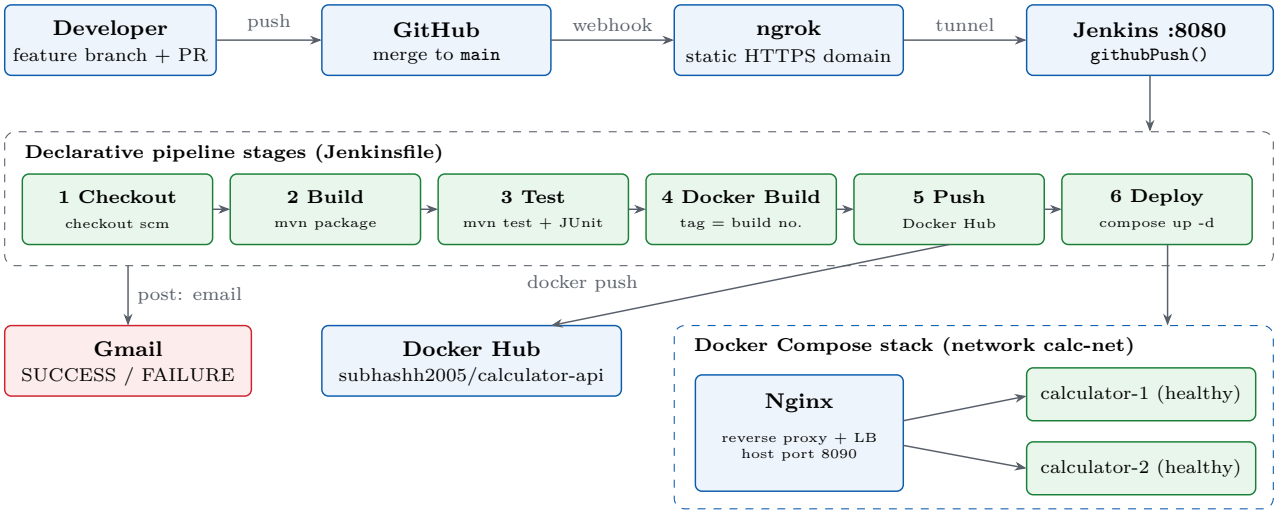


Figure 1: Pipeline architecture. A merged PR reaches Jenkins through a GitHub webhook over an ngrok tunnel. The six stages publish `:<build no.>` to Docker Hub and redeploy two API replicas behind Nginx, and the result is emailed.

2. Application and testing

The service uses Spring Boot 4.1 on Java 21 with Maven, and is layered as controller → service → DTO. The endpoints `GET /api/v1/add`, `/subtract`, `/multiply` and `/divide` take query parameters `a` and `b`. For example, `add?a=2&b=3` returns `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`. A single `@RestControllerAdvice` turns every client error into HTTP 400 with a structured body (`timestamp`, `status`, `error`, `message`, `path`). It covers division by zero, a missing or non-numeric parameter, NaN/Infinity, and an overflowing result. Division by zero needs an explicit check, because with `double`, `1/0` silently yields `Infinity`. Spring Boot Actuator exposes `/actuator/health`, which every health check in the system uses.

There are 28 JUnit 5 tests. `CalculatorServiceTest` has 19 unit tests, mostly parameterized, covering every operation, divide-by-zero, non-finite input and overflow. `CalculatorControllerTest` has 9 integration tests: `@SpringBootTest` with MockMvc hits the real endpoints, the three 400 cases and the health probe. `mvn test` writes JUnit XML to `target/surefire-reports`, and Jenkins publishes it.

3. CI/CD pipeline (Jenkinsfile)

1. **Checkout:** `checkout scm` of the merged commit. The implicit checkout is disabled so this stage owns it.
2. **Build:** `mvn -B clean package -DskipTests` using the Jenkins-managed tools `JDK-21` and `Maven-3.9`.
3. **Test:** `mvn -B test`, then `junit 'target/surefire-reports/*.xml'`. A failing test fails the stage, and every later stage is skipped.
4. **Docker Build:** builds the multi-stage image as `subhashh2005/calculator-api:`*build-number* and `:latest`.
5. **Push:** `credentials('dockerhub-creds')` exposes a masked user and access token. The stage runs `docker login -password-stdin` and pushes both tags. The Docker Hub user name comes from the credential, so it is not hard-coded anywhere.
6. **Deploy:** `docker compose up -d -wait -remove-orphans` with `IMAGE_TAG` set to the build number. `-wait` fails the stage unless every health check passes. A `curl` smoke test through Nginx then checks `/actuator/health` and that `2+3` returns `5.0`.

Trigger. `githubPush()` fires on the webhook. GitHub cannot reach `localhost:8080`, so the webhook targets a static ngrok domain that tunnels to Jenkins. `pollSCM('H/5 * * * *')` remains as a fallback. Build #4 was *"Started by GitHub push"* within seconds of merging PR #1 (Fig. 2). **Notifications.** The `post` block sends a SUCCESS email (build number, image tag, link) or a FAILURE email with `build.log` attached, through Gmail SMTP over SSL with an app password. The recipients are set in Jenkins, not in the repository. **Idempotence.** `disableConcurrentBuilds()`,

the fixed Compose project name `calculator`, and `up -d` (which recreates only changed containers) make re-runs safe. Nginx stayed up through every redeploy (Fig. 6).

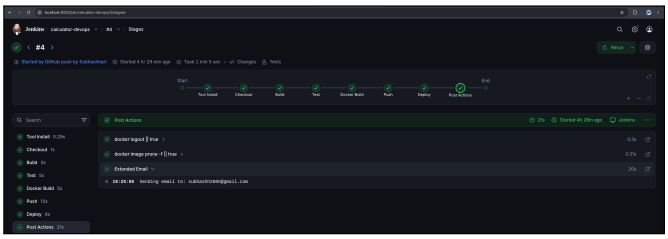


Figure 2: Build #4, started by GitHub push: all six stages and the post actions are green.

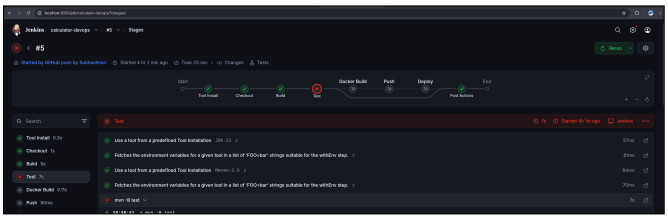


Figure 3: Build #5 (deliberately broken test, PR #2): Test fails, so Docker Build, Push and Deploy are skipped.

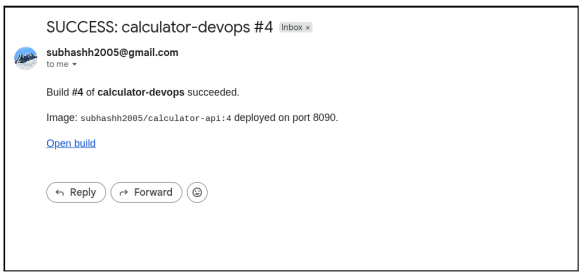


Figure 4: SUCCESS email for build #4.

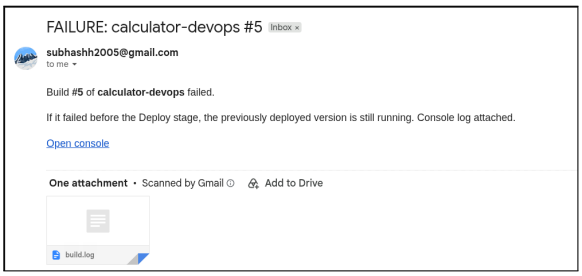


Figure 5: FAILURE email for build #5, with the console log attached.

4. Containerization (Dockerfile)

The **build stage** (`maven:3.9-eclipse-temurin-21`) copies `pom.xml` first and runs `mvn dependency:go-offline` before adding `src/`. Dependencies therefore form a cached layer, and a code-only change rebuilds only the last step. The **runtime stage** (`eclipse-temurin:21-jre-alpine`) copies just the JAR and runs it as the non-root user `app`. It declares `EXPOSE 8080` and has a `HEALTHCHECK` that calls `/actuator/health` with BusyBox `wget`. The image is about 95 MB compressed. `JAVA_OPTS` is honoured by an `exec java` entrypoint, and `.dockerignore` keeps `target/`, `.git`, IDE files and `.env` out of the build context. A single `docker run -p 8085:8080 subhashh2005/calculator-api` answers on the mapped port.

5. Compose deployment and verification

`docker-compose.yml` defines:

- the bridge network `calc-net`;
- the `calculator` service with **two replicas**, an Actuator health check, `restart: unless-stopped`, and the environment variables `SPRING_PROFILES_ACTIVE` and `JAVA_OPTS`;
- an **Nginx** reverse proxy, the only published port (8090), which starts only once the API is healthy and has its own health check.

The image and tag are parameters (`DOCKER_IMAGE`, `IMAGE_TAG`), so Jenkins deploys exactly the image it pushed. Nginx uses `upstream { server calculator:8080 resolve; }` with Docker's DNS (`127.0.0.11`). It therefore round-robins across both replicas and follows their new IPs after a redeploy, and the `X-Upstream` response header shows which replica answered. The deployment was verified with `docker ps`, calls to every endpoint through Nginx including the 400 cases, and the redeploy from `:4` to `:6`.

```
$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES                   IMAGE                              STATUS                    PORTS
calculator-calculator-1   subhashh2005/calculator-api:6    Up About a minute (healthy)   8080/tcp
calculator-calculator-2   subhashh2005/calculator-api:6    Up About a minute (healthy)   8080/tcp
calculator-nginx-1        nginx:1.28-alpine                Up 10 hours (healthy)         0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
```

Figure 6: `docker ps` after build #6: two healthy replicas running `:6` and the Nginx proxy on host port 8090. Nginx was not recreated by the redeploy.

```
$ curl -s "localhost:8090/api/v1/add?a=2&b=3"; echo
curl -s "localhost:8090/api/v1/subtract?a=10&b=4"; echo
curl -s "localhost:8090/api/v1/multiply?a=4&b=5"; echo
curl -s "localhost:8090/api/v1/divide?a=10&b=4"; echo
curl -s -w "  [HTTP %{http_code}]\n" "localhost:8090/api/v1/divide?a=1&b=0"
curl -s -w "  [HTTP %{http_code}]\n" "localhost:8090/api/v1/add?a=2"
curl -s -w "  [HTTP %{http_code}]\n" "localhost:8090/api/v1/add?a=abc&b=1"
curl -s "localhost:8090/actuator/health"; echo
{"operation":"add","a":2.0,"b":3.0,"result":5.0}
{"operation":"subtract","a":10.0,"b":4.0,"result":6.0}
{"operation":"multiply","a":4.0,"b":5.0,"result":20.0}
{"operation":"divide","a":10.0,"b":4.0,"result":2.5}
{"timestamp":"2026-10-30T17:32:51.130575051Z","status":400,"error":"Bad Request","message":"Division by zero is not allowed","path":"/api/v1/divide"}  [HTTP 400]
{"timestamp":"2026-10-30T17:32:51.138788375Z","status":400,"error":"Bad Request","message":"Missing required parameter 'b'","path":"/api/v1/add"}  [HTTP 400]
{"timestamp":"2026-10-30T17:32:51.146556252Z","status":400,"error":"Bad Request","message":"Parameter 'a' must be a number","path":"/api/v1/add"}  [HTTP 400]
{"groups":["liveness","readiness"],"status":"UP"}
$
```

Figure 7: API responses through Nginx: the four operations, three HTTP 400 errors and the health endpoint.



Figure 8: The `X-Upstream` header alternates between the two replicas, showing round-robin load balancing.

6. Version control

`main` has 18 commits: 15 Conventional-Commit changes (`feat`, `test`, `build`, `ci`, `docs`, `fix`, `chore`) and 3 pull-request merges. **PR #1** (`docs/dockerhub-link`) was the first webhook-triggered deploy. **PR #2** (`test/failing-build`) deliberately broke a test to prove that a failure blocks deployment. **PR #3** reverted it, and build #6 went green. `.gitignore` excludes `target/`, IDE files, logs and `.env`. The Docker Hub token and Gmail app password exist only in the Jenkins credential store.

7. Challenges faced

Challenge	Cause and resolution
Jenkins `apt` install hung	IPv6 connections to `pkg.jenkins.io` time out on this network. Used `wget -4` and `apt -o Acquire::ForceIPv4=true`, and added `-Djava.net.preferIPv4Stack=true` to Jenkins with a systemd override so plugin and Maven downloads work.
`NO_PUBKEY 7198F4B714ABFC68`	The Jenkins repository signing key had been rotated. Switched to `jenkins.io-2026.key` after checking that its fingerprint matched the missing key.
Parent POM not found	Spring Initializr labels the version `4.1.2.RELEASE`, but the Maven artifact is `4.1.2`. Fixed the parent version.
`@AutoConfigureMockMvc` not found	Spring Boot 4 moved the MockMvc auto-configuration into `spring-boot-starter-webmvc-test` (package `org.springframework.boot.webmvc.test.autoconfigure`).
First build failed at compile time	*"Tool type jdk does not have an install of JDK-21"*: the tool had been registered as `jdk-21`, and names are case-sensitive. Renamed it. The Jenkinsfile itself had been checked earlier with Jenkins' declarative linter.
Nginx sent every request to one replica	A variable in `proxy_pass` re-resolved DNS but stuck to one address. An `upstream` with `server ... resolve` (Nginx 1.28) balances both replicas and tracks new IPs.
Port 80 already in use	Another service holds port 80. Nginx is published on 8090, Jenkins keeps 8080, and the API replicas are not published at all.
GitHub cannot reach Jenkins	Jenkins runs on a laptop. An ngrok static domain forwards the webhook, and Poll SCM is kept as a fallback in case the tunnel is down.

Demonstration. Start the ngrok tunnel. Make a small change on a feature branch, open a PR and merge it. Then show Jenkins starting by itself, the six stages turning green, the new tag on Docker Hub, the SUCCESS email, and `docker ps` showing the replicas on the new tag while Nginx keeps serving.